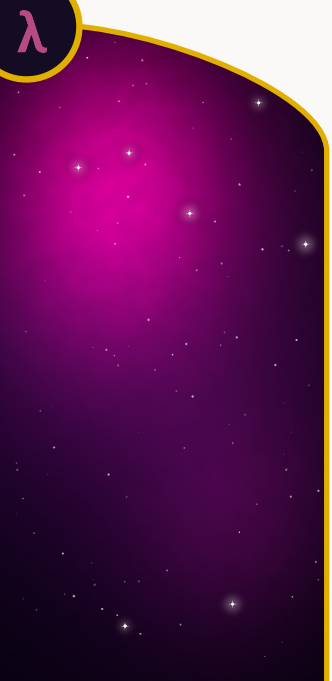


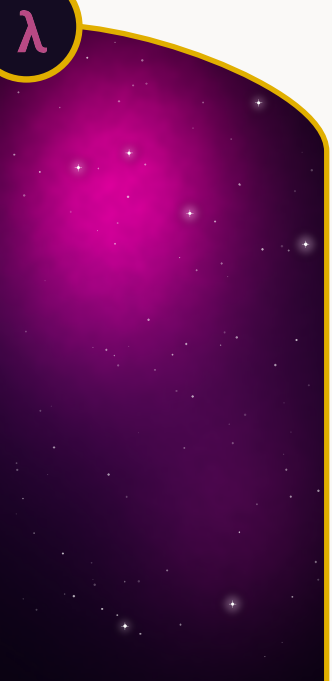
L11B: Metacircular Evaluator I

CS1101S: Programming Methodology

Martin Henz

30th October, 2026

Version 1.1 — Built: 2026-10-26 23:27:12 SGT

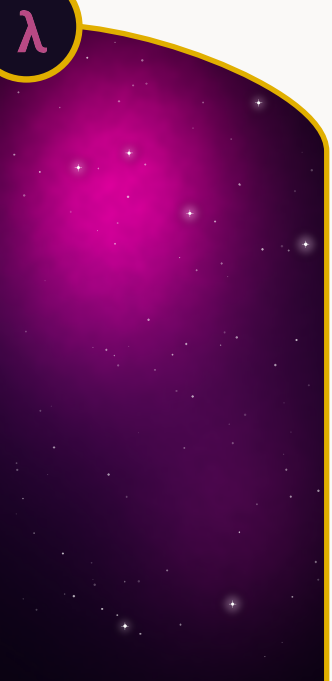


On metacircular evaluators

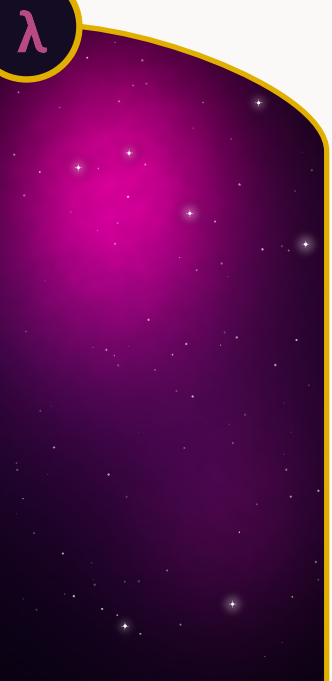
Calculator language

Conditionals and sequences

Names and assignment



Importance of metalinguistic abstraction



Importance of metalinguistic abstraction

The most fundamental idea in programming

The evaluator, which determines the meaning of statements and expressions in a programming language, is just another program.

Importance of metalinguistic abstraction

The most fundamental idea in programming

The evaluator, which determines the meaning of statements and expressions in a programming language, is just another program.

- Lecture L7B introduced interpreters as language processors.

Importance of metalinguistic abstraction

The most fundamental idea in programming

The evaluator, which determines the meaning of statements and expressions in a programming language, is just another program.

- Lecture L7B introduced interpreters as language processors.
- SICPy Section 4.1 takes a different approach: the interpreter is written in the language that it interprets.

Importance of metalinguistic abstraction

The most fundamental idea in programming

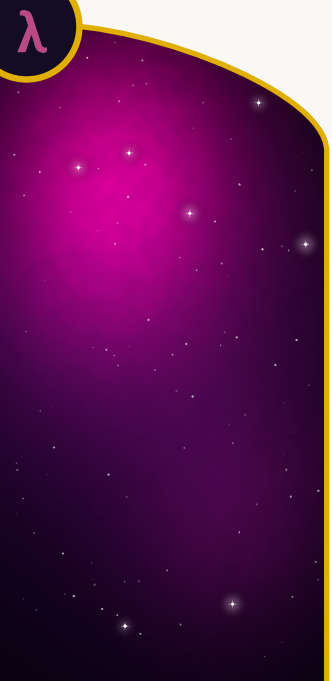
The evaluator, which determines the meaning of statements and expressions in a programming language, is just another program.

- Lecture L7B introduced interpreters as language processors.
- SICPy Section 4.1 takes a different approach: the interpreter is written in the language that it interprets.
- This lecture and the next follow SICPy Section 4.1: The Metacircular Evaluator.

A high-level evaluator

Metacircular evaluator is high-level

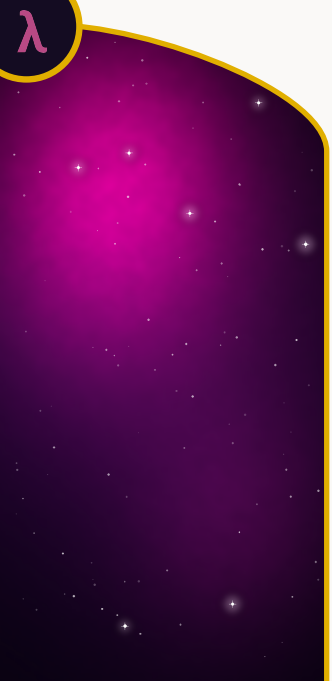
It freely uses recursion and higher-order functions. Explains the environment model of SICPy Chapter 3 (Lecture L9A).



The most beautiful program ever written

Gerald Jay Sussman, CS1101S Guest Lecture 2020/21, 25:35

"The most beautiful program ever written, in my opinion, is the Eval/Apply Interpreter."



The most beautiful program ever written

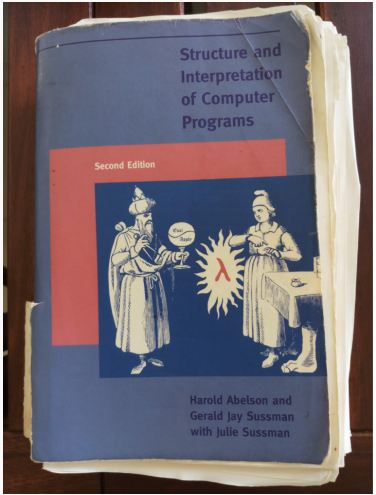
Gerald Jay Sussman, CS1101S Guest Lecture 2020/21, 25:35

"The most beautiful program ever written, in my opinion, is the Eval/Apply Interpreter."

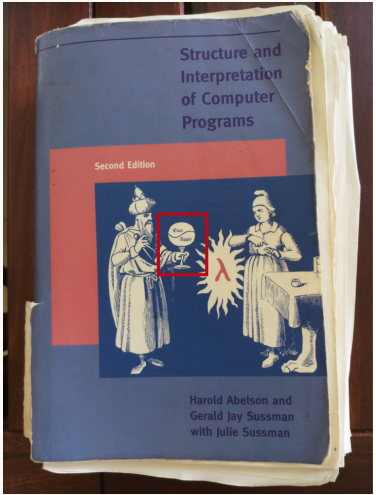
Why is this beautiful?

It is beautiful in the same way as Maxwell's Equations on electromagnetism are beautiful: It conveys deep insights in a few lines.

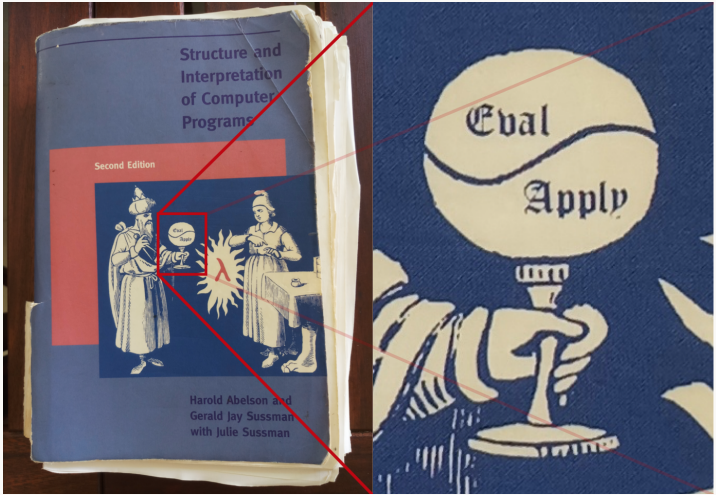
SICP

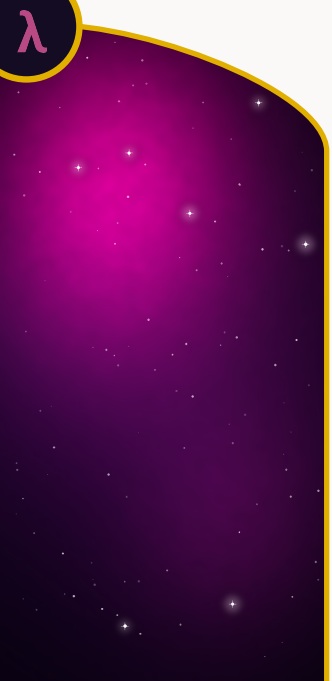

Structure and Interpretation of Computer Programs
Second Edition
λ
Harold Abelson and Gerald Jay Sussman with Julie Sussman

SICP: What is that in the middle?



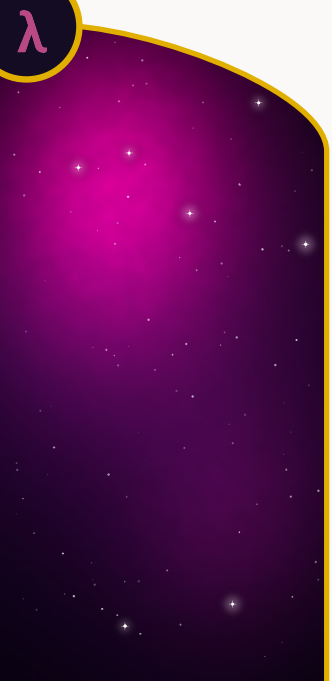
SICP: The importance of Eval/Apply





Our task

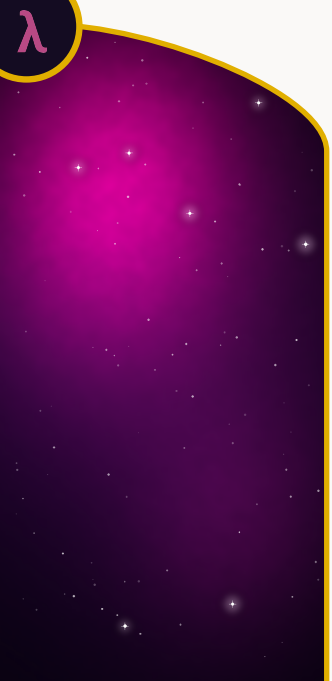
Task



Our task

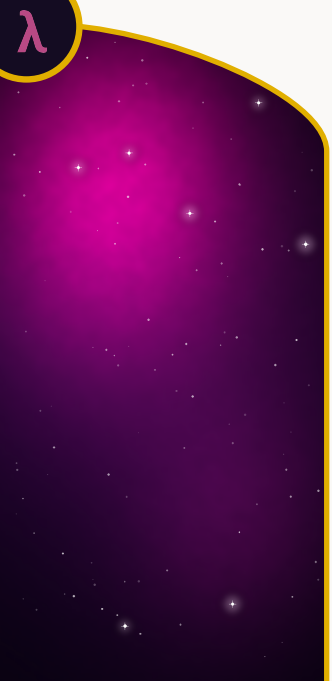
Task

We want to write an interpreter in Python that can execute programs that are written in Python



Common starting point: parsing

- Start with program string
- Parse program string, resulting in a parse tree
- Pass parse tree to `evaluate` function

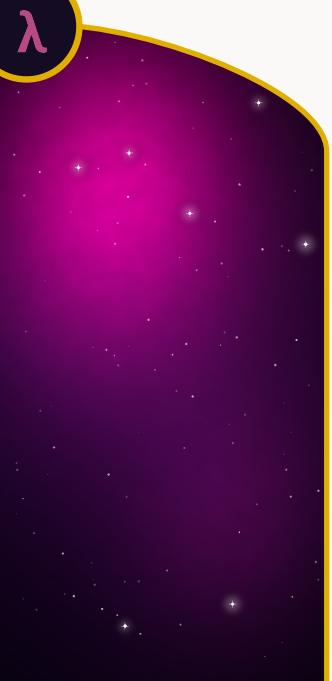


On metacircular evaluators

Calculator language

Conditionals and sequences

Names and assignment



Calculator language in Backus-Naur Form (BNF)

Calculator language in Backus-Naur Form (BNF)

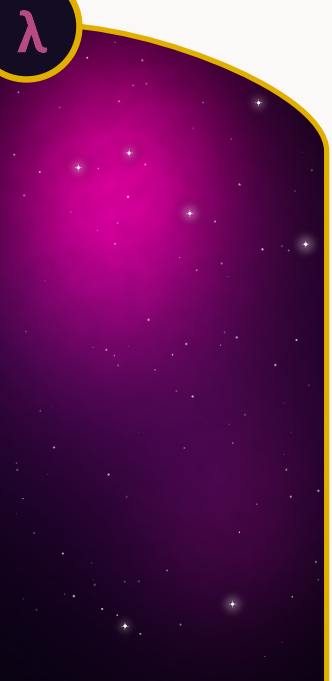
stmt	::=	*expr*	expression statement
expr	::=	*expr bin-op expr*	binary operator combination
	|	*number*	number literal
	|	(*expr*)	parenthesised expression
bin-op	::=	+ | - | * | /	binary operator

Calculator language in Backus-Naur Form (BNF)

stmt	::=	*expr*	expression statement
expr	::=	*expr bin-op expr*	binary operator combination
	|	*number*	number literal
	|	(*expr*)	parenthesised expression
bin-op	::=	+ | - | * | /	binary operator

Example:

```
1.4 + 2.3 / 70.4
```



Parsing the calculator language

Parsing the calculator language

To run the program

```
1.4 + 2.3 / 70.4
```



Parsing the calculator language

To run the program

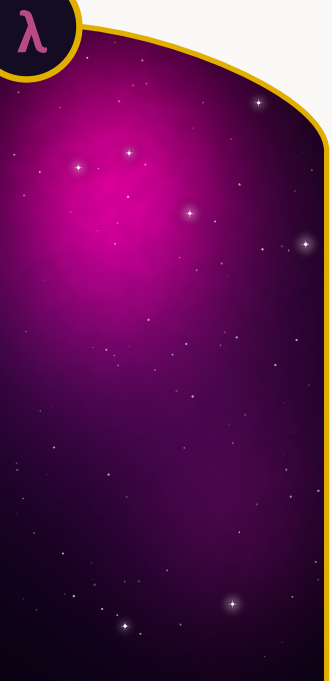
```
1.4 + 2.3 / 70.4
```



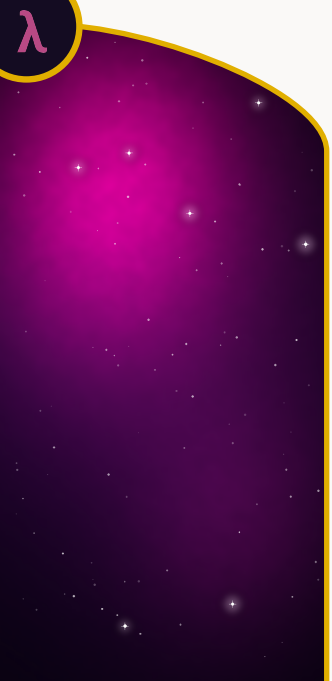
we first parse it:

```
print(parse('1.4 + 2.3 / 70.4'))
```





Parsing the calculator language (cont.)



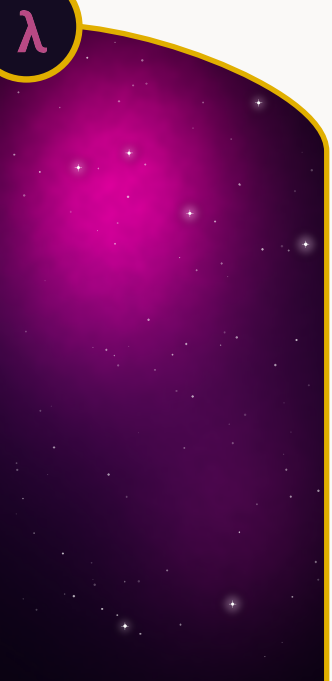
Parsing the calculator language (cont.)

The result is a syntax tree, a linked list with a tag:

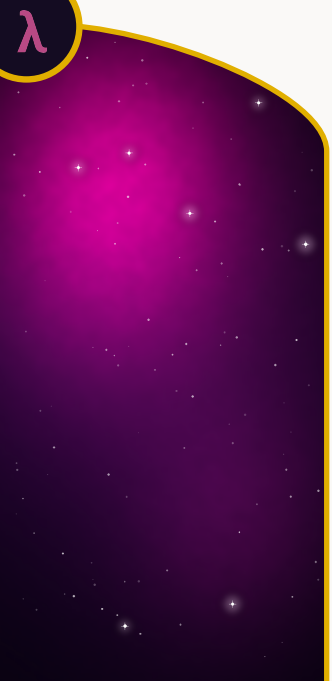
Parsing the calculator language (cont.)

The result is a syntax tree, a linked list with a tag:

```
tree = llist(
    'binary_operator_combination',
    '+',
    llist('literal', 1.4),
    llist(
        'binary_operator_combination',
        '/',
        llist('literal', 2.3),
        llist('literal', 70.4)
    )
)
print(parse('1.4 + 2.3 / 70.4') == tree)
```



Syntax of literals



Syntax of literals

Example: `parse('...38...')` ⇒
`...llist('literal', 38)...`

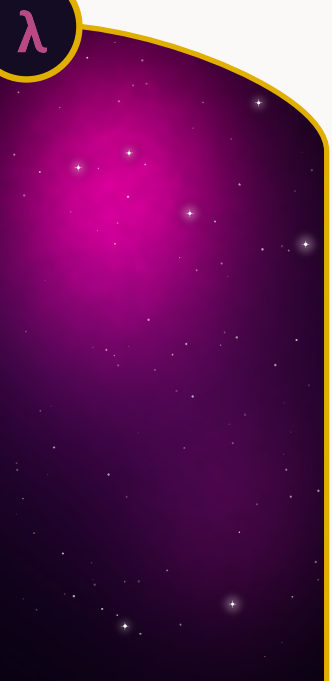
Syntax of literals

Example: `parse('...38...')` ⇒
`...llist('literal', 38)...`

```
# syntax predicate
def is_literal(component):
    return is_tagged_list(component, 'literal')

def is_tagged_list(component, the_tag):
    return is_pair(component) and head(component) ==
    the_tag

# selector
def literal_value(component):
    return head(tail(component))
```



Syntax of operator combinations

Syntax of operator combinations

parse('... 2.3 + 70.4 ...') ⇒

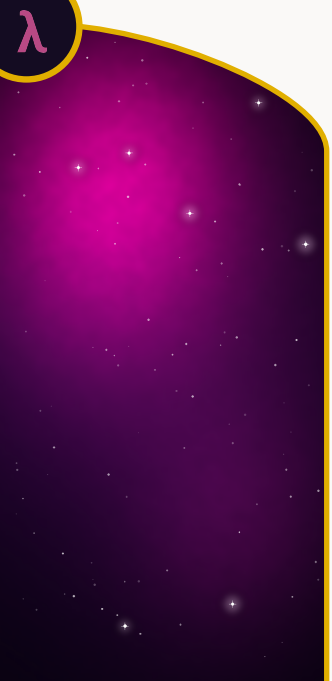
```
... llist('binary_operator_combination', '+',
          llist('literal', 2.3),
          llist('literal', 70.4)) ...
```

Syntax of operator combinations

parse('... 2.3 + 70.4 ...') ⇒

```
... llist('binary_operator_combination', '+',
          llist('literal', 2.3),
          llist('literal', 70.4)) ...
```

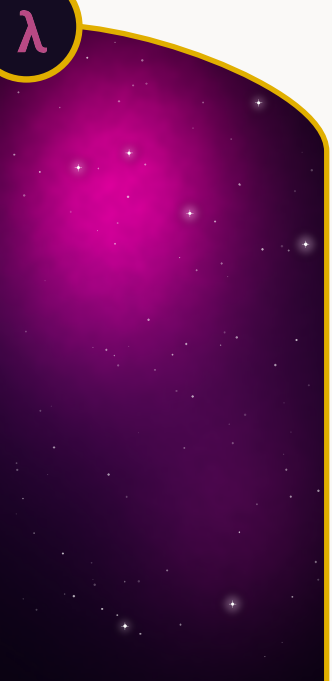
```
# syntax predicate
def is_operator_combination(component):
    return is_tagged_list(
        component, 'binary_operator_combination')
# selectors
def operator_symbol(component):
    return llist_ref(component, 1)
def first_operand(component):
    return llist_ref(component, 2)
def second_operand(component):
    return llist_ref(component, 3)
```



Evaluating programs in the calculator language

Evaluating programs in the calculator language

```
def evaluate(component):
    if is_literal(component):
        return literal_value(component)
    elif is_operator_combination(component):
        operator = operator_symbol(component)
        operands = llist(first_operand(component),
                         second_operand(component))
        return apply(operator, llist_of_values(operands))
    else:
        return error('unknown syntax', component)
```



Evaluating the operands

Evaluating the operands

```
def llist_of_values(exps):
    return map(evaluate, exps)
```

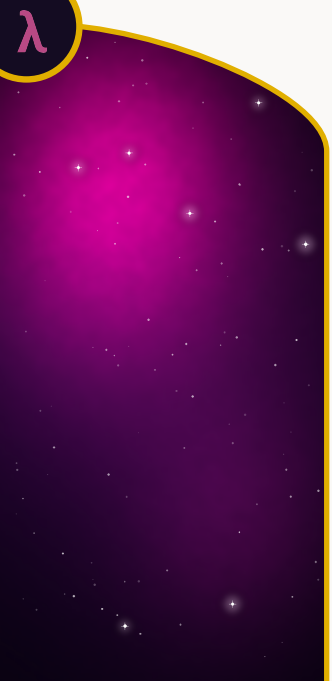


Evaluating the operands

```
def llist_of_values(exps):
    return map(evaluate, exps)
```



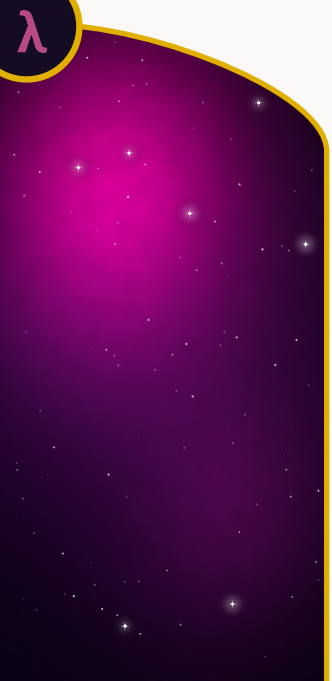
`evaluate` and `llist_of_values` call each other: they follow the structure of the syntax tree.



Applying operators

Applying operators

```
def apply(operator, operands):
    first_op = head(operands)
    second_op = head(tail(operands))
    return (first_op + second_op if operator == '+' else
            first_op - second_op if operator == '-' else
            first_op * second_op if operator == '*' else
            first_op / second_op if operator == '/' else
            error('unknown operator -- apply', operator))
```



Putting it together

Putting it together

```
def evaluate_program(program):
    evaluate(program)

# evaluate_program discards the value,
# so we use evaluate to see it
print(evaluate(parse('1.4 + 2.3 / 70.4')))
```



Putting it together

```
def evaluate_program(program):
    evaluate(program)

# evaluate_program discards the value,
# so we use evaluate to see it
print(evaluate(parse('1.4 + 2.3 / 70.4')))
```

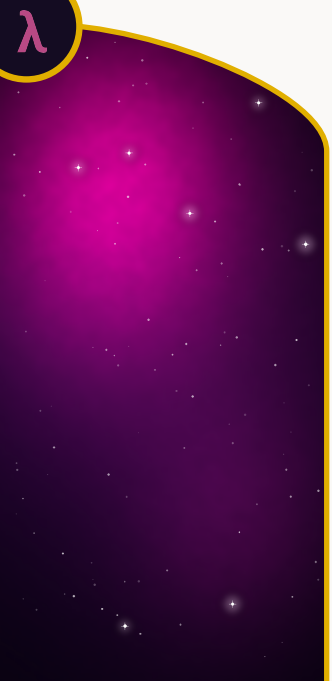
Parse the program string, then evaluate the syntax tree.

Putting it together

```
def evaluate_program(program):
    evaluate(program)

# evaluate_program discards the value,
# so we use evaluate to see it
print(evaluate(parse('1.4 + 2.3 / 70.4')))
```

Parse the program string, then evaluate the syntax tree.
`evaluate_program` runs a program and discards its value.

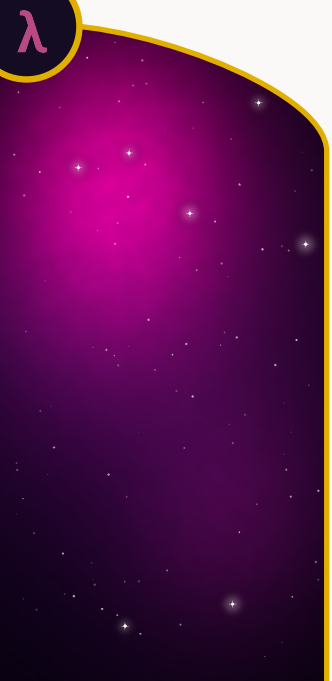


On metacircular evaluators

Calculator language

Conditionals and sequences

Names and assignment



Adding booleans, conditionals, and sequences

Adding booleans, conditionals, and sequences

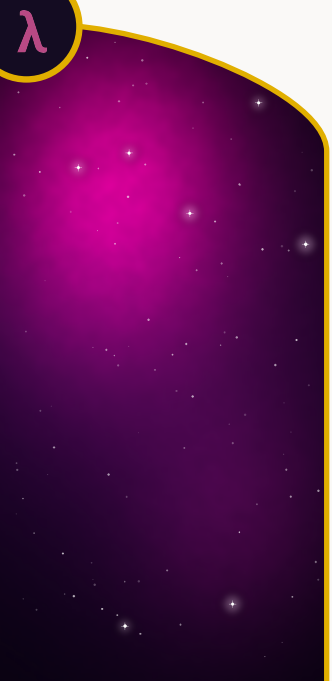
stmt	::=	$\cdots$	
	$\mid$	$stmt_1$ ... $stmt_n$	statement sequence
expr	::=	$\cdots$	
	$\mid$	`True` $\mid$ `False`	boolean literals
	$\mid$	$expr_2$ `if` $expr_1$ `else` $expr_3$	conditional expression

Adding booleans, conditionals, and sequences

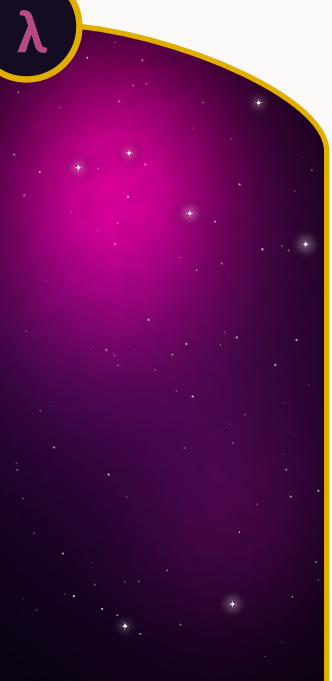
stmt	::=	$\cdots$	
	$\mid$	$stmt_1$ ... $stmt_n$	statement sequence
expr	::=	$\cdots$	
	$\mid$	`True` $\mid$ `False`	boolean literals
	$\mid$	$expr_2$ `if` $expr_1$ `else` $expr_3$	conditional expression

Example:

```
8 + 34
1 + 2 if True else 17
```



Parsing conditionals: an example



Parsing conditionals: an example

`parse('42 if True else 17')` returns

Parsing conditionals: an example

parse('42 if True else 17') returns

```
tree = llist(
    'conditional_expression',
    llist('literal', True),
    llist('literal', 42),
    llist('literal', 17)
)
print(parse('42 if True else 17') == tree)
```

Parsing conditionals: an example

`parse('42 if True else 17')` returns

```
tree = llist(
    'conditional_expression',
    llist('literal', True),
    llist('literal', 42),
    llist('literal', 17)
)
print(parse('42 if True else 17') == tree)
```

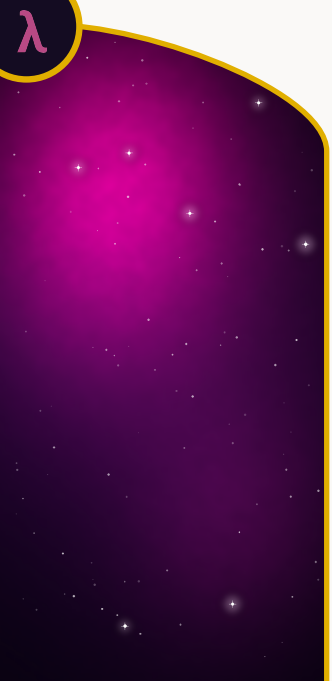
- Syntax predicate: `is_conditional`

Parsing conditionals: an example

`parse('42 if True else 17')` returns

```
tree = llist(
    'conditional_expression',
    llist('literal', True),
    llist('literal', 42),
    llist('literal', 17)
)
print(parse('42 if True else 17') == tree)
```

- Syntax predicate: `is_conditional`
- Selectors: `conditional_predicate`, `conditional_consequent`, `conditional_alternative`



Syntax functions: conditionals

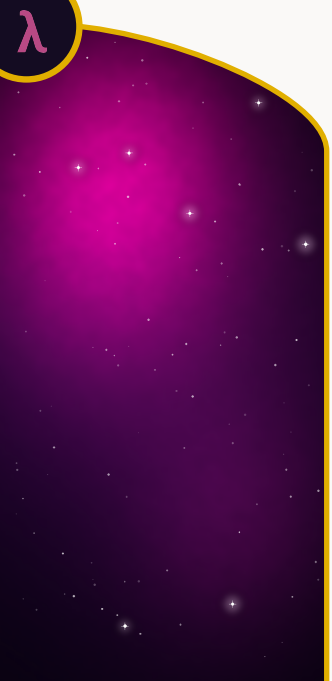
Syntax functions: conditionals

```
def is_conditional(component):
    return is_tagged_list(
        component, 'conditional_expression')

def conditional_predicate(component):
    return llist_ref(component, 1)

def conditional_consequent(component):
    return llist_ref(component, 2)

def conditional_alternative(component):
    return llist_ref(component, 3)
```



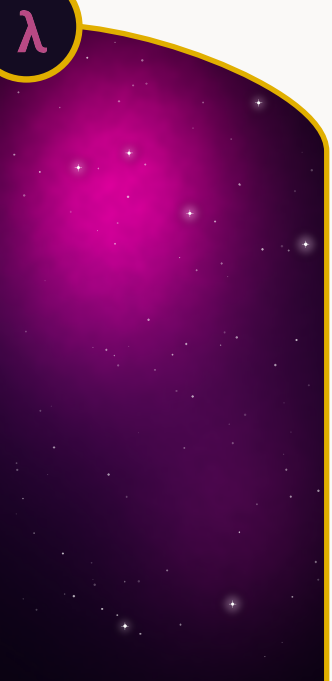
Evaluating conditionals

Evaluating conditionals

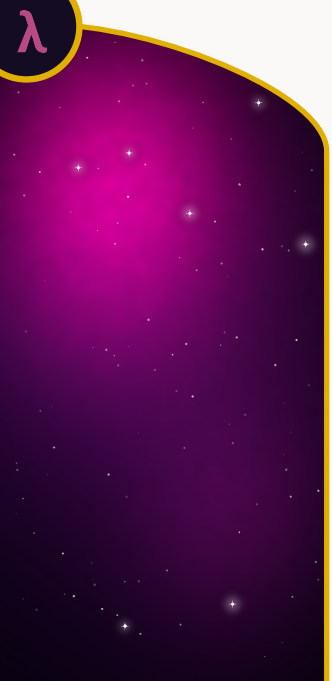
Example: `42 if True else 17`

```
def evaluate(component):
    if is_literal(component):
        ...
    elif is_conditional(component):
        return eval_conditional(component)
    ...

def eval_conditional(component):
    if
    is_truthy(evaluate(conditional_predicate(component))):
        return evaluate(conditional_consequent(component))
    else:
        return
    evaluate(conditional_alternative(component))
```

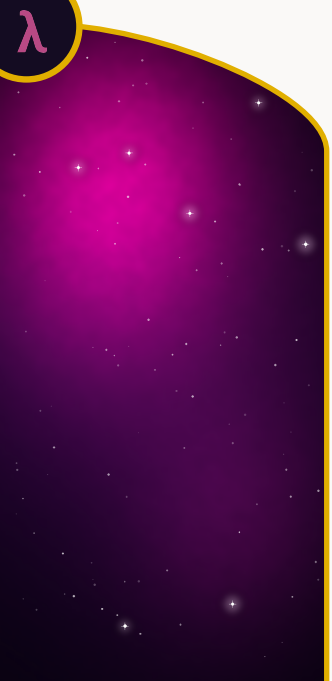


Evaluating conditionals (cont.)



Evaluating conditionals (cont.)

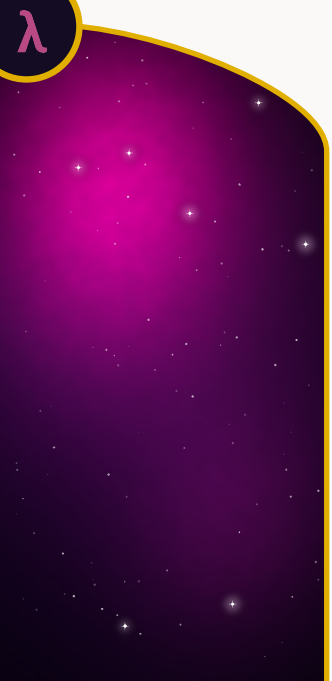
We explain conditionals in the interpreted language using conditionals in the interpreting language.



Evaluating conditionals (cont.)

We explain conditionals in the interpreted language using conditionals in the interpreting language.

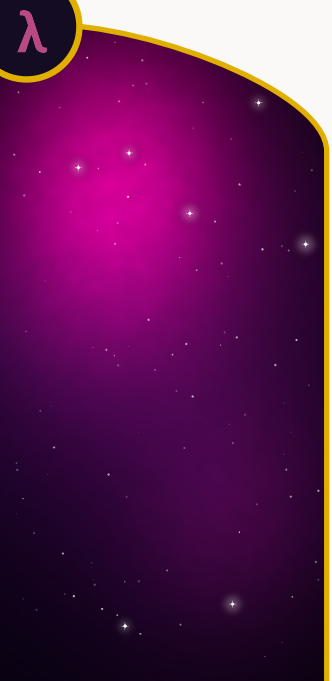
- Easy to read



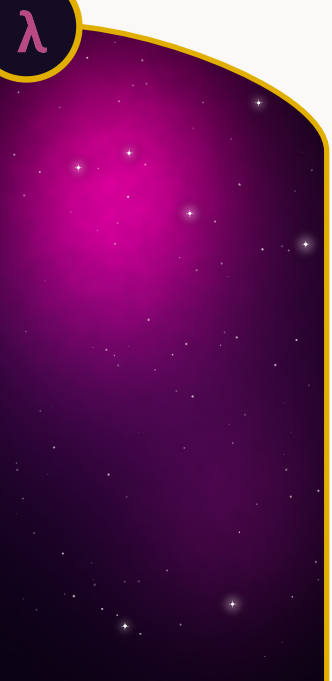
Evaluating conditionals (cont.)

We explain conditionals in the interpreted language using conditionals in the interpreting language.

- Easy to read
- Philosophically somewhat questionable



Parsing sequences: an example



Parsing sequences: an example

`parse('1\n2\n3')` returns

Parsing sequences: an example

`parse('1\n2\n3')` returns

```
tree = llist('sequence',
       llist(llist('literal', 1),
             llist('literal', 2),
             llist('literal', 3)))
print(parse('1\n2\n3') == tree)
```

Parsing sequences: an example

`parse('1\n2\n3')` returns

```
tree = llist('sequence',
       llist(llist('literal', 1),
             llist('literal', 2),
             llist('literal', 3)))
print(parse('1\n2\n3') == tree)
```

- Syntax predicate: `is_sequence`

Parsing sequences: an example

`parse('1\n2\n3')` returns

```
tree = llist('sequence',
      llist(llist('literal', 1),
            llist('literal', 2),
            llist('literal', 3)))
print(parse('1\n2\n3') == tree)
```

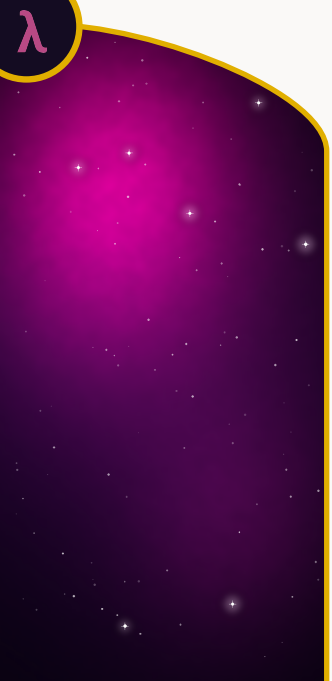
- Syntax predicate: `is_sequence`
- Selectors: `sequence_statements`, `first_statement`, `rest_statements`

Parsing sequences: an example

`parse('1\n2\n3')` returns

```
tree = llist('sequence',
      llist(llist('literal', 1),
            llist('literal', 2),
            llist('literal', 3)))
print(parse('1\n2\n3') == tree)
```

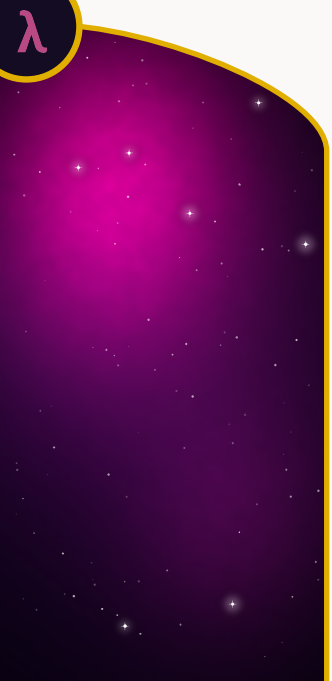
- Syntax predicate: `is_sequence`
- Selectors: `sequence_statements`, `first_statement`, `rest_statements`
- More syntax predicates: `is_empty_sequence`, `is_last_statement`



Syntax functions: sequences

Syntax functions: sequences

```
def is_sequence(stmt):
    return is_tagged_list(stmt, 'sequence')
def sequence_statements(stmt):
    return head(tail(stmt))
def first_statement(stmts):
    return head(stmts)
def rest_statements(stmts):
    return tail(stmts)
def is_empty_sequence(stmts):
    return is_none(stmts)
def is_last_statement(stmts):
    return is_none(tail(stmts))
```



Evaluating sequences

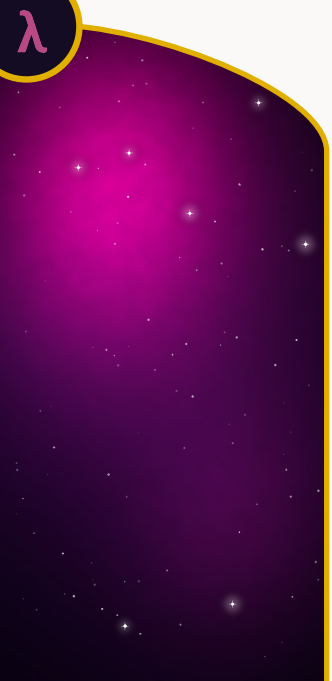
Evaluating sequences

Example:

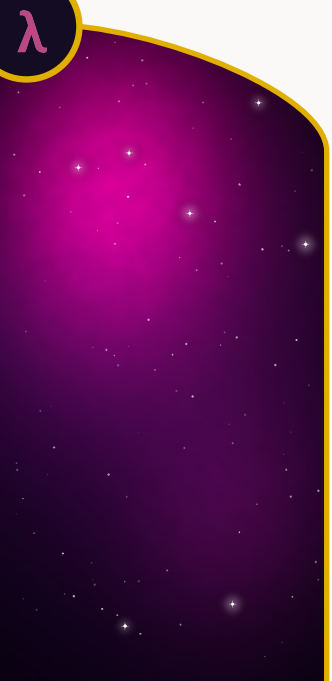
```
1 + 2
3 * 4
5 - 6
```



```
def eval_sequence(stmts):
    if is_empty_sequence(stmts):
        return None
    elif is_last_statement(stmts):
        return evaluate(first_statement(stmts))
    else:
        evaluate(first_statement(stmts))
        return eval_sequence(rest_statements(stmts))
```

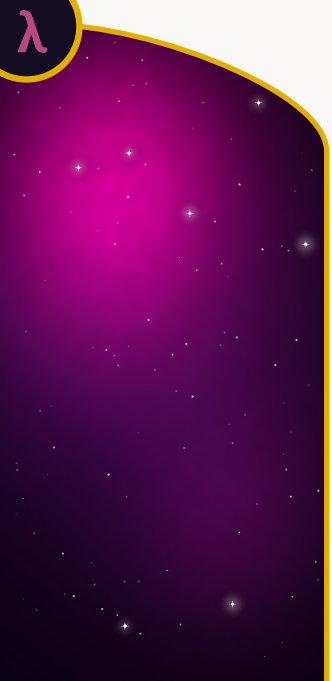


Evaluating sequences (cont.)



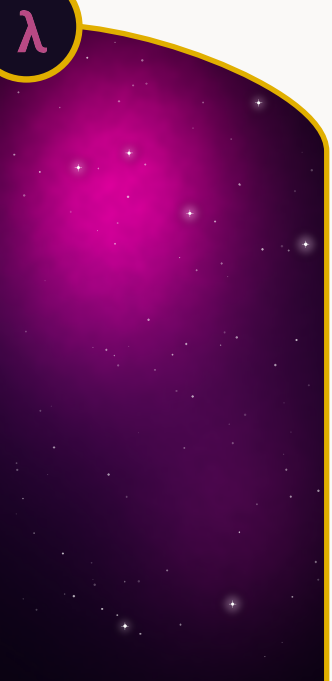
Evaluating sequences (cont.)

We explain sequences in the interpreted language using sequences in the interpreting language.



Evaluating sequences (cont.)

We explain sequences in the interpreted language using sequences in the interpreting language. The statements are evaluated one after the other, in order.

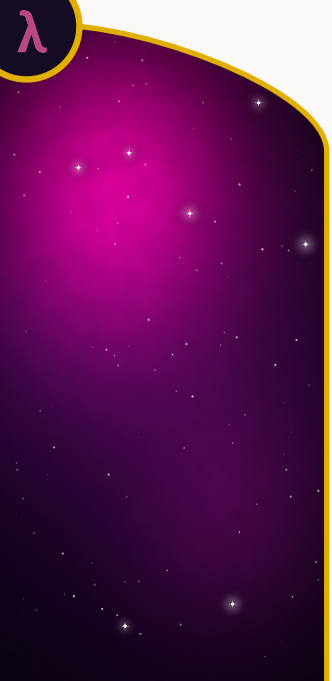


On metacircular evaluators

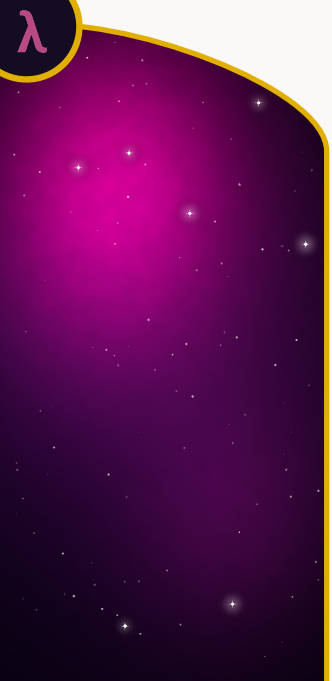
Calculator language

Conditionals and sequences

Names and assignment



Adding names and assignment



Adding names and assignment

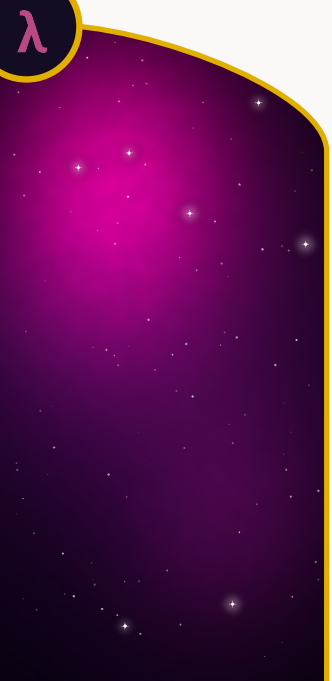
stmt	::=	$\cdots$	
	|	*name* = *expr*	assignment
expr	::=	$\cdots$	
	|	*name*	name

Adding names and assignment

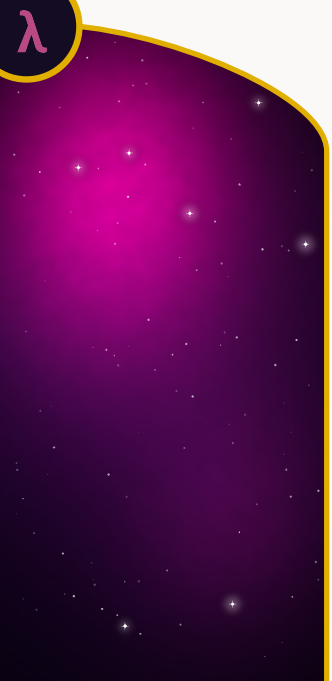
stmt	::=	⋯	
	|	*name* = *expr*	assignment
expr	::=	⋯	
	|	*name*	name

Example:

```
y = 4
x = y + 7
x * 2
```



Parsing assignments: an example



Parsing assignments: an example

`parse('size = 2\n5 * size')` returns

Parsing assignments: an example

`parse('size = 2\n5 * size')` returns

```
tree = llist('sequence',
       llist(llist('assignment',
                   llist('name', 'size'),
                   llist('literal', 2)),
             llist('binary_operator_combination', '*',
                   llist('literal', 5),
                   llist('name', 'size'))))
print(parse('size = 2\n5 * size') == tree)
```

Parsing assignments: an example

`parse('size = 2\n5 * size')` returns

```
tree = llist('sequence',
       llist(llist('assignment',
                   llist('name', 'size'),
                   llist('literal', 2)),
             llist('binary_operator_combination', '*',
                   llist('literal', 5),
                   llist('name', 'size'))))
print(parse('size = 2\n5 * size') == tree)
```

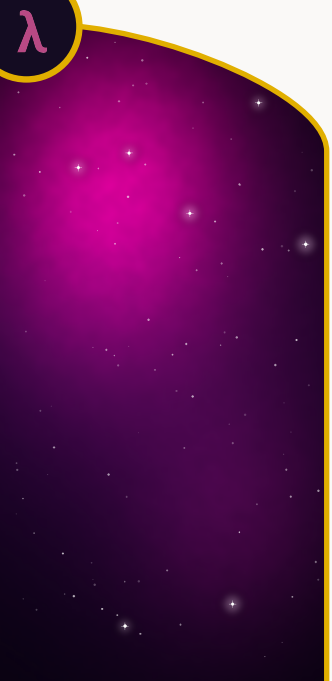
- Syntax predicates: `is_assignment`, `is_name`

Parsing assignments: an example

`parse('size = 2\n5 * size')` returns

```
tree = llist('sequence',
       llist(llist('assignment',
                   llist('name', 'size'),
                   llist('literal', 2)),
             llist('binary_operator_combination', '*',
                   llist('literal', 5),
                   llist('name', 'size'))))
print(parse('size = 2\n5 * size') == tree)
```

- Syntax predicates: `is_assignment`, `is_name`
- Selectors: `assignment_symbol`, `assignment_value_expression`, `symbol_of_name`



Syntax functions: names and assignments

Syntax functions: names and assignments

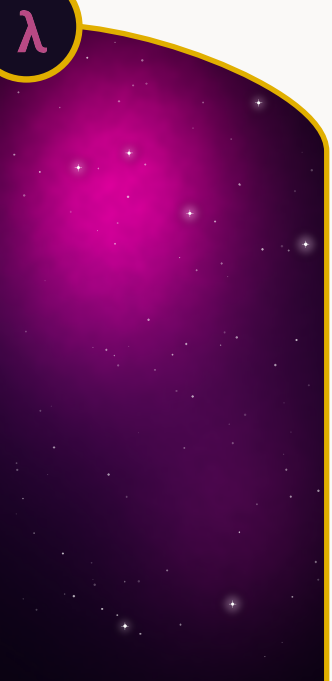
```
def is_name(component):
    return is_tagged_list(component, 'name')

def symbol_of_name(component):
    return head(tail(component))

def is_assignment(component):
    return is_tagged_list(component, 'assignment')

def assignment_symbol(component):
    return symbol_of_name(head(tail(component)))

def assignment_value_expression(component):
    return head(tail(tail(component)))
```



Frames and environments

Frames and environments

```
def make_frame(symbols, values):
    return pair(symbols, values)

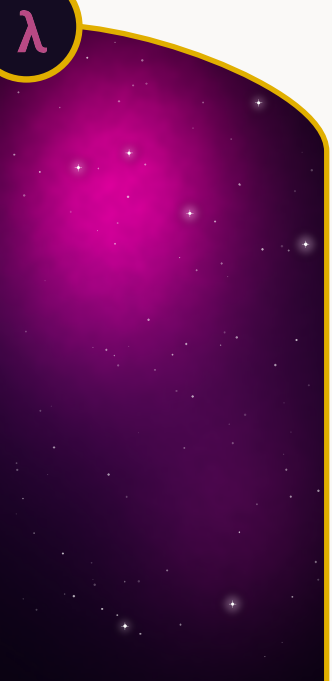
def frame_symbols(frame):
    return head(frame)

def frame_values(frame):
    return tail(frame)

def enclosing_environment(env):
    return tail(env)

def first_frame(env):
    return head(env)

the_empty_environment = None
```



Extending environments and frames

Extending environments and frames

```
def add_binding_to_frame(symbol, val, frame):
    set_head(frame, pair(symbol, head(frame)))
    set_tail(frame, pair(val, tail(frame)))

def extend_environment(symbols, vals, base_env):
    if length(symbols) == length(vals):
        return pair(make_frame(symbols, vals), base_env)
    else:
        return error('arity mismatch')

the_global_environment = extend_environment(
    None, None, the_empty_environment)
```

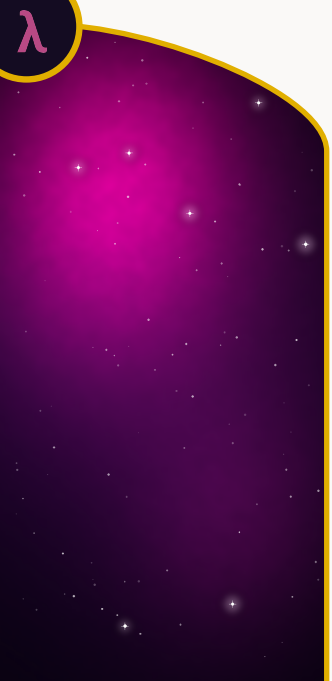
Extending environments and frames

```
def add_binding_to_frame(symbol, val, frame):
    set_head(frame, pair(symbol, head(frame)))
    set_tail(frame, pair(val, tail(frame)))

def extend_environment(symbols, vals, base_env):
    if length(symbols) == length(vals):
        return pair(make_frame(symbols, vals), base_env)
    else:
        return error('arity mismatch')

the_global_environment = extend_environment(
    None, None, the_empty_environment)
```

A frame is a pair of linked lists: symbols and values.

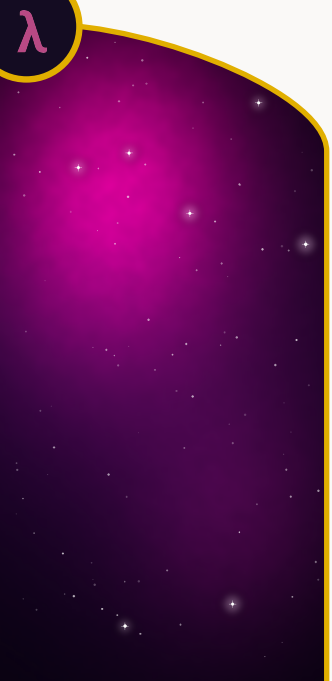


Looking up a name's value

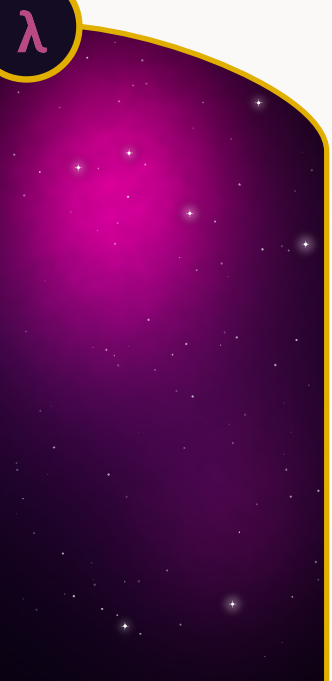
Looking up a name's value

Example: `lookup_symbol_value('x', env)`

```
def lookup_symbol_value(symbol, env):
    def env_loop(env):
        def scan(symbols, vals):
            if is_none(symbols):
                return env_loop(
                    enclosing_environment(env))
            elif symbol == head(symbols):
                return head(vals)
            else:
                return scan(tail(symbols),
                            tail(vals))
        if env is the_empty_environment:
            return error('unbound name', symbol)
        else:
            frame = first_frame(env)
            return scan(frame_symbols(frame),
                        frame_values(frame))
    return env_loop(env)
```



Looking up a name's value (cont.)

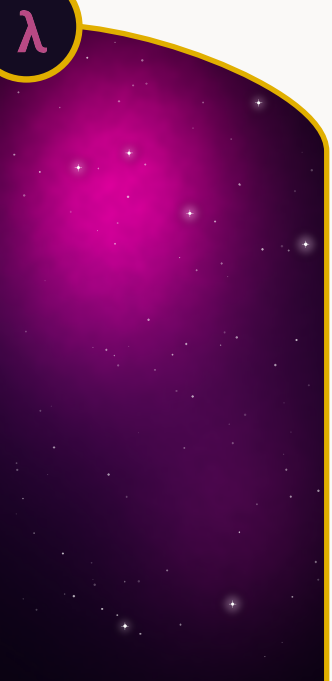


Looking up a name's value (cont.)

If the name is not in the first frame, the search continues in the enclosing environment.

Looking up a name's value (cont.)

If the name is not in the first frame, the search continues in the enclosing environment. If it reaches the empty environment, the name is unbound.

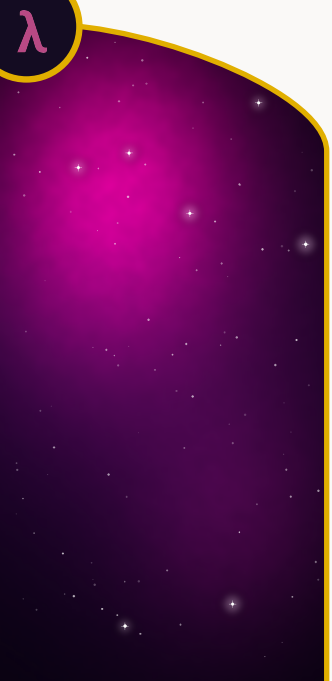


Assigning a value to a name

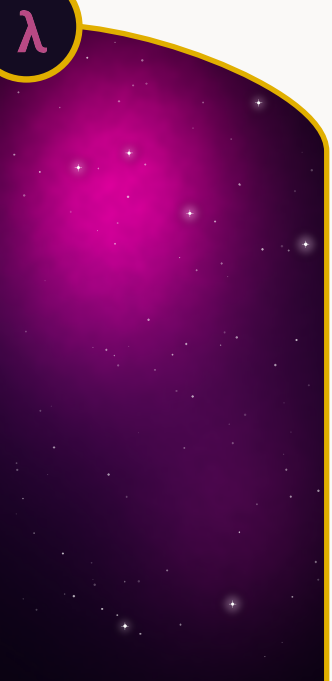
Assigning a value to a name

Example: `x = 8 + 34` ⇒
`assign_symbol_value('x', 42, env)`

```
def assign_symbol_value(symbol, val, env):
    def scan(symbols, vals):
        if is_none(symbols):
            if env is the_global_environment:
                add_binding_to_frame(
                    symbol, val, first_frame(env))
            else:
                assign_symbol_value(
                    symbol, val,
                    enclosing_environment(env))
        elif symbol == head(symbols):
            set_head(vals, val)
        else:
            scan(tail(symbols), tail(vals))
    frame = first_frame(env)
    scan(frame_symbols(frame),
         frame_values(frame))
```

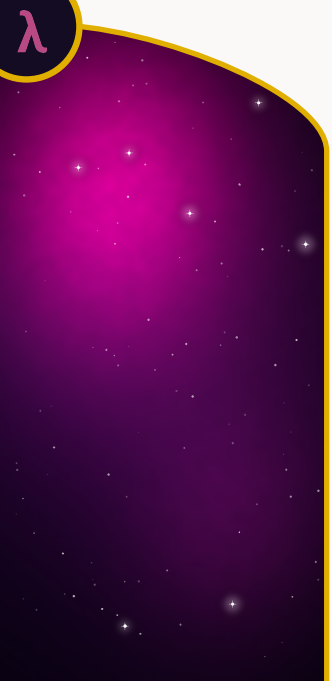


Assigning a value to a name (cont.)



Assigning a value to a name (cont.)

- A name's first assignment adds a binding to the global frame

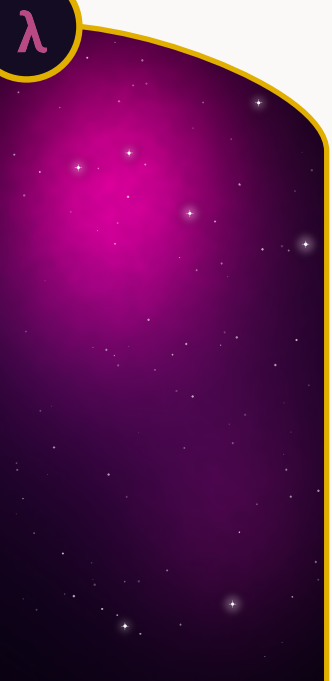


Assigning a value to a name (cont.)

- A name's first assignment adds a binding to the global frame
- A later assignment to the same name updates the binding in place

Assigning a value to a name (cont.)

- A name's first assignment adds a binding to the global frame
- A later assignment to the same name updates the binding in place
- Next lecture: functions introduce more frames

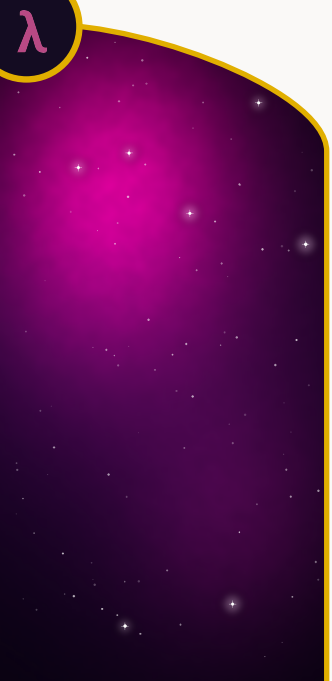


Evaluating the operands

Evaluating the operands

```
def llist_of_values(exps, env):
    return map(lambda exp: evaluate(exp, env), exps)

def eval_operator_combination(component, env):
    operands = llist(first_operand(component),
                     second_operand(component))
    return apply(operator_symbol(component),
                 llist_of_values(operands, env))
```



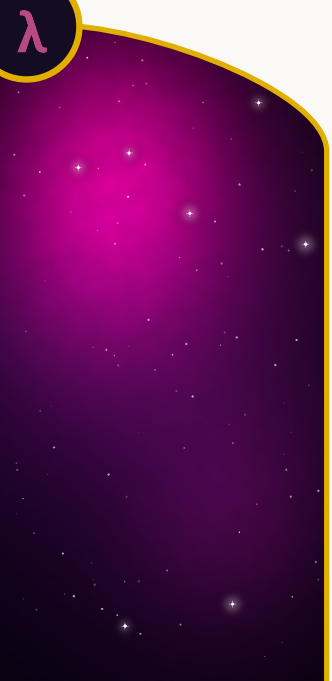
Evaluating conditionals

Evaluating conditionals

Example: `42 if True else 17`

```
def is_truthy(x):
    if is_boolean(x):
        return x
    else:
        return error('boolean expected, received', x)

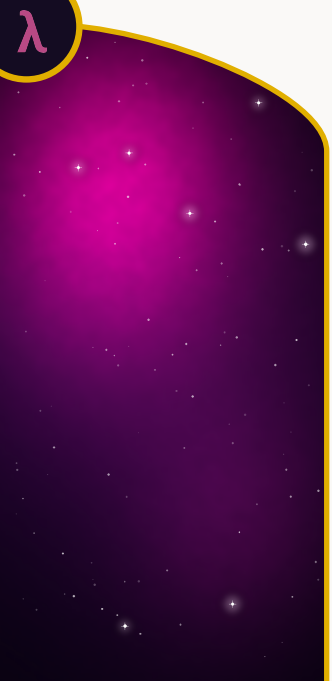
def eval_conditional(component, env):
    pred = conditional_predicate(component)
    if is_truthy(evaluate(pred, env)):
        cons = conditional_consequent(component)
        return evaluate(cons, env)
    else:
        alt = conditional_alternative(component)
        return evaluate(alt, env)
```



Evaluating sequences

Evaluating sequences

```
def eval_sequence(stmts, env):
    if is_empty_sequence(stmts):
        return None
    elif is_last_statement(stmts):
        return evaluate(first_statement(stmts), env)
    else:
        evaluate(first_statement(stmts), env)
        return eval_sequence(
            rest_statements(stmts), env)
```



Evaluating assignments

Evaluating assignments

Example: `x = y + 7`

```
def eval_assignment(component, env):
    expr = assignment_value_expression(component)
    value = evaluate(expr, env)
    assign_symbol_value(assignment_symbol(component),
                        value, env)
    return value
```

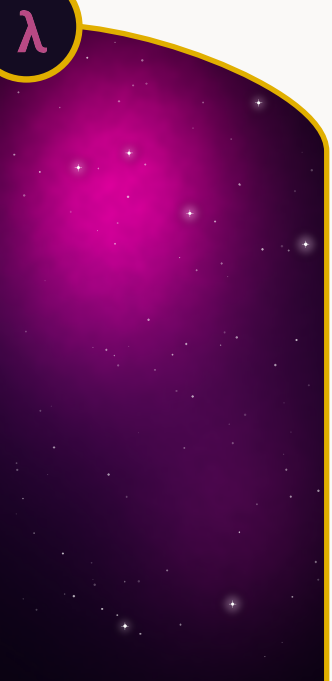


Evaluating assignments

Example: `x = y + 7`

```
def eval_assignment(component, env):
    expr = assignment_value_expression(component)
    value = evaluate(expr, env)
    assign_symbol_value(assignment_symbol(component),
                        value, env)
    return value
```

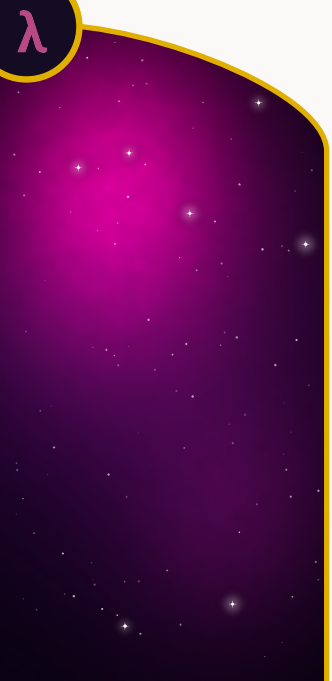
The expression is evaluated first; then the name is assigned.



The evaluator with environments

The evaluator with environments

```
def evaluate(component, env):
    if is_literal(component):
        return literal_value(component)
    elif is_name(component):
        return lookup_symbol_value(
            symbol_of_name(component), env)
    elif is_operator_combination(component):
        return eval_operator_combination(component, env)
    elif is_conditional(component):
        return eval_conditional(component, env)
    elif is_sequence(component):
        return eval_sequence(
            sequence_statements(component), env)
    elif is_assignment(component):
        return eval_assignment(component, env)
    else:
        return error('unknown syntax', component)
```

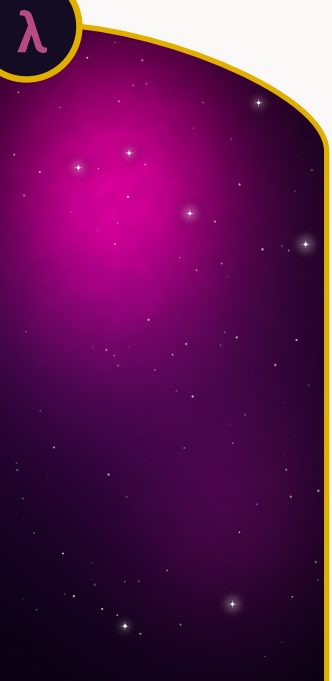


Evaluating a program

Evaluating a program

```
def evaluate_program(program):
    evaluate(program, the_global_environment)

program = parse('y = 4\nx = y + 7\nx * 2')
# evaluate_program discards the value,
# so we use evaluate to see it
print(evaluate(program, the_global_environment))
```



Evaluating programs interactively

Evaluating programs interactively

```
from repl import set_evaluator, set_program_text

def parse_and_evaluate(program):
    return evaluate(parse(program),
    the_global_environment)

set_evaluator(parse_and_evaluate)
set_program_text("x = 2 + 3\nx * 4")
```

Evaluating programs interactively

```
from repl import set_evaluator, set_program_text

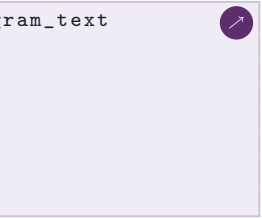
def parse_and_evaluate(program):
    return evaluate(parse(program),
    the_global_environment)

set_evaluator(parse_and_evaluate)
set_program_text("x = 2 + 3\nx * 4")
```



- The Repl tab calls `parse_and_evaluate` with every program that you run there, and shows the value that `evaluate` returns

Evaluating programs interactively

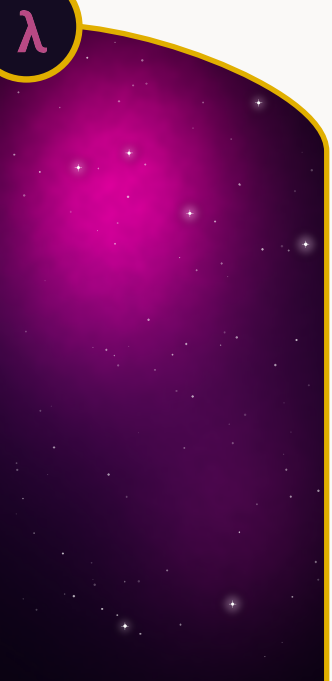


```
from repl import set_evaluator, set_program_text

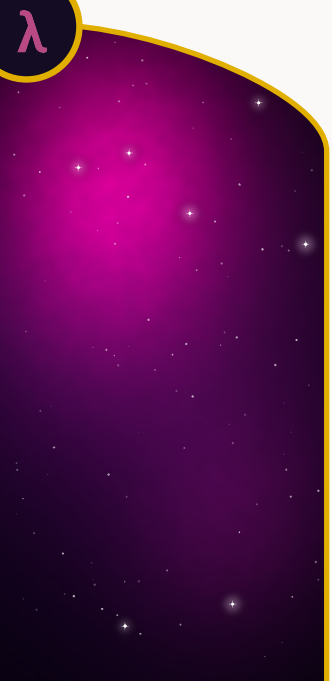
def parse_and_evaluate(program):
    return evaluate(parse(program),
    the_global_environment)

set_evaluator(parse_and_evaluate)
set_program_text("x = 2 + 3\nx * 4")
```

- The Repl tab calls `parse_and_evaluate` with every program that you run there, and shows the value that `evaluate` returns
- Assigned names stay in the global environment for later programs

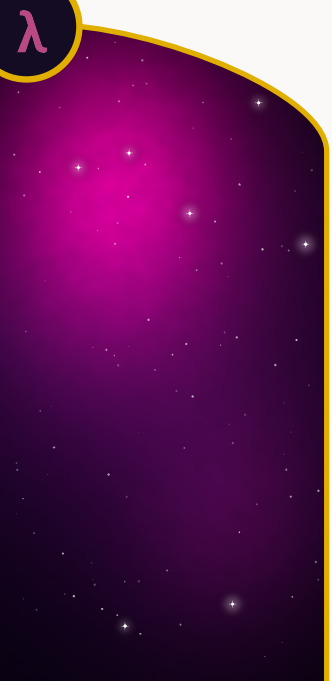


Summary



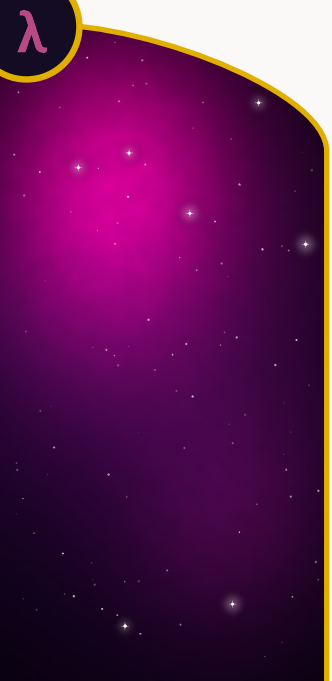
Summary

- The evaluator is just another program



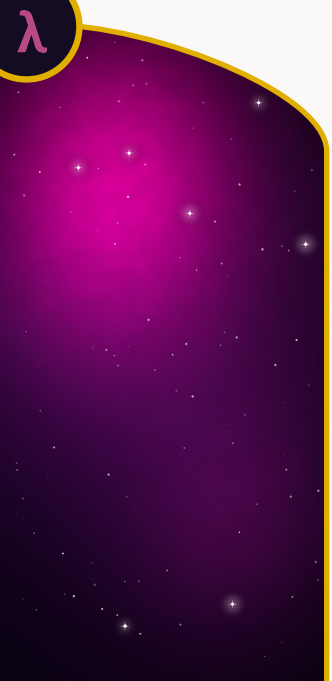
Summary

- The evaluator is just another program
- `evaluate` dispatches on the syntax tree; `apply` combines the operand values



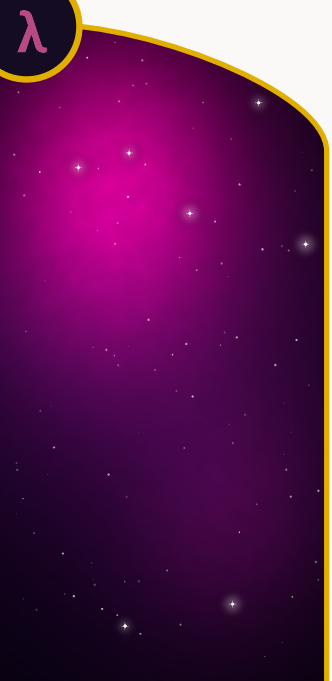
Summary

- The evaluator is just another program
- `evaluate` dispatches on the syntax tree; `apply` combines the operand values
- Conditionals and sequences are explained using conditionals and sequences of the interpreting language



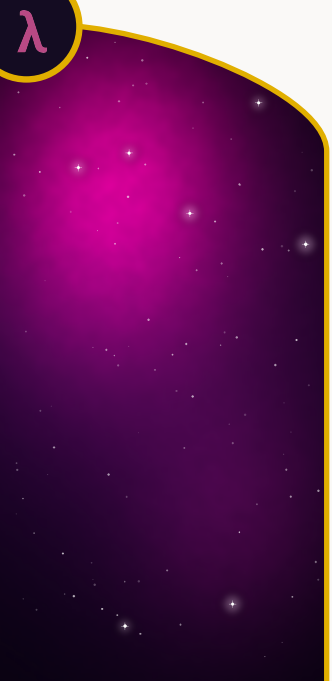
Summary

- The evaluator is just another program
- `evaluate` dispatches on the syntax tree; `apply` combines the operand values
- Conditionals and sequences are explained using conditionals and sequences of the interpreting language
- Assignment: environments as linked lists of frames



Summary

- The evaluator is just another program
- `evaluate` dispatches on the syntax tree; `apply` combines the operand values
- Conditionals and sequences are explained using conditionals and sequences of the interpreting language
- Assignment: environments as linked lists of frames
- Next lecture: functions, return statements, and lazy evaluation



Check-In time!

Don't go around in circles: evaluate!

Look for Check-In I11B in the Source Academy!